Claude Learning Plan: compliance report

Sr. Web Developer, AdVise I Stream demo app I Verified on October 21, 2026 with Node 26

30	4	1	2	3	3	1	44
Done	Partial	Needs you	Pending	Changed by decision	Not done (optional)	Out of scope	Exercises

Verified: 565 unit tests, 112 Playwright tests with axe (light and dark themes, 320 px), 87 back-end tests, type check, lint with the Vue accessibility rules, license policy, traceability (47 acceptance criteria, 17 states), Storybook build, performance budget (initial JavaScript 80.5 kB gzip of 100), plugin validation in strict mode, and an accessibility and performance audit through the Chrome DevTools MCP on 10 pages, desktop and mobile (accessibility 100, best practices 100, LCP under 0.5 s, CLS 0). Not verified: anything that needs a person (see the status Needs you and Pending).

Week 1 (Oct 19 to 23): foundations and skills

Exercise proposed in the plan	Status	Evidence or what is left
Confirm Claude Code works with the Gorilla Logic licence	Done	Confirmed by you: the work is done with Gorilla Logic's Claude.
Set up the sandbox: Vue 3 + TypeScript, Vite, Vue Router, Pinia, Vitest + Vue Test Utils, ESLint with the Vue accessibility rules, Storybook, Playwright	Done	All present. ESLint uses eslint-plugin-vue plus eslint-plugin-vuejs-accessibility, and vue/no-v-html is an error. npm run check is green.
Write the mock API with the five behaviours: preview with holds, apply with partial failure, locked-field rejection, a 2,000-row list, a rate limit	Changed by decision	Kept: preview with held items and reasons, apply with retryable partial failure (including provider rate-limit failures), idempotency. The 2,000-row list and the inline locked-field edit were removed on your decision (the status filter on Campaigns replaced them); field locks remain in the real campaign change form. mock-backend/ and the Laravel API. The 2,000-row list, the locked-field edit and the rate limit are now written as acceptance criteria in tickets #1 and #2 to be built on the Laravel API with ticket-to-spec; not built yet.
Application shell: routing, sign-in and session, typed API client with one place for error handling, consistent loading, empty and error states	Done	src/app, src/shared/api/client.ts, Sanctum sign-in with roles. Every list and tab has loading, empty and error states.
Small design language: colour, spacing and type tokens and base components (button, table, form field, status badge) with their states	Done	tokens.json generates the CSS variables; 15 Ds* components (including DsTabs and DsAccordion) with stories per state, light and dark.
Work through plan mode and permissions	Done	Declared by you: done while building the project from your instructions.
CLAUDE.md with the conventions (tokens, states, partial failure, keyboard, one test per criterion, no v-html, no real accounts)	Done	CLAUDE.md, plus rules in .claude/rules/ (including UX and information design) and the product map.

Skill that turns a written screen spec into a component, its states, one test per criterion and one story per state	Done	.claude/skills/screen-from-spec; npm run traceability checks every criterion and state. ticket-to-spec starts one step earlier: from a GitHub issue to the spec, then to the screen, the API or both.
Write the acceptance criteria yourself before running the skill	Pending	Pending: the criteria still have to come from your product owner. Two example tickets with criteria (GitHub issues #1 Campaign Explorer, UI and API; #2 Bulk budget change API) were drafted to rehearse the flow, and the skill ticket-to-spec turns an issue into a spec and stops for your approval. The specs in docs/specs/ (47 criteria) remain illustrative drafts.
Run the skill on two specs: a 300-item bulk change with preview, held items, one approval and 12 retryable failures; a template field kept within 30 characters after placeholders are filled	Done	Bulk budget change and Ad copy on Campaigns, built by the skill's procedure. The live run of /screen-from-spec on a new mockup is rehearsed in prompts/01.
Optional: a list of 2,000 rows with filters, sorting and a status column that stays fast, with inline editing that can be rejected as locked	Changed by decision	Removed on your decision. Campaigns now has a status filter (Active, Paused, Limited, Ended) with counts; performance is covered by the bundle budget and the DevTools audit.
Review everything it generated and start the review log	Done	docs/learning-plan/REVIEW_LOG.md, with findings from independent reviewer runs.
Attend the Gorilla Logic Construct walkthrough	Pending	Pending: you do not have access to Construct yet.

Week 2 (Oct 26 to 30): subagents

Exercise proposed in the plan	Status	Evidence or what is left
Build a read-only reviewer subagent that checks a screen against the design language, accessibility and the acceptance criteria	Done	screen-reviewer, plus frontend-code-reviewer, backend-code-reviewer (one per stack), a11y-reviewer and write-path-guardian. All read-only.
Plant five weak screens on purpose	Done	frontend/review-fixtures/weak-screens/: hard-coded styles, no error state, v-html, loads all rows and filters in the browser, drops failed items.
Tune the reviewer until it catches all five and log what you changed	Done	The reviewer judged all five FAIL naming each planted defect; npm run review:fixtures catches 5 of 5 by script. Logged.
Optional: a fourth spec (compliance administration or UTM) generated with the skill and checked by the reviewer	Not done (optional)	Not built. The OEM compliance mockup and the prompt to build it live are ready in prompts/10. The Admin page already lists OEM rulebooks with effective dates.
Automated accessibility checks with axe in component tests, and a keyboard-only pass by hand on each screen	Partial	axe runs in unit tests, Storybook and Playwright, and the DevTools accessibility tree and Lighthouse run on 10 pages. Keyboard flows are automated in Playwright. The hand pass is yours.
Make generation and review one repeatable step: the skill generates the screen, then asks the reviewer	Done	Steps 9 and 10 of screen-from-spec call the reviewer and the log.

Optional: compare each state's Storybook story against the design language and log drift	Not done (optional)	Not done.
A 15-minute usability check with a colleague on the bulk-approval screen	Needs you	Needs a person. Script and table in docs/learning-plan/USABILITY_CHECK.md.

Week 3 (Nov 2 to 6): hooks, MCP and plugins

Exercise proposed in the plan	Status	Evidence or what is left
Hook that runs lint (with the Vue accessibility rules), the type check and the affected tests after every component edit	Done	format.mjs (Prettier, ESLint) and verify.mjs (vue-tsc, vitest related, screen check); proven by tools/hooks.spec.ts.
Hook that blocks edits to environment and secrets files	Done	guard-files.mjs blocks .env, keys, certificates, credentials and lockfiles; guard-bash.mjs blocks destructive database commands.
Connect the Playwright MCP, walk the screens and draft end-to-end tests with axe, then check each locator by hand and log what you rejected	Partial	Playwright and Chrome DevTools MCP servers are in .mcp.json (pinned, free) and pre-approved in .claude/settings.json. The 112 end-to-end tests were written as Playwright code, not drafted through the MCP. The live walk is in prompts/08.
A 2,000-row list with a performance budget (Lighthouse CI or a timing test) that fails the build	Changed by decision	The list was removed on your decision. The budget remains: npm run budget fails when initial JavaScript exceeds 100 kB gzip, and npm run audit:devtools fails below accessibility 100, best practices 95, LCP 2.5 s or CLS 0.1.
Optional: point one screen at a small Laravel endpoint that returns a partial result	Not done (optional)	Not done: the batch screens run on the mock. The rest of the app runs on the real Laravel API (accounts, campaigns, approvals, changelog, simulated campaigns with edit and delete). Ticket #2 specifies the Laravel bulk budget endpoints (preview, apply with partial failure, retry); the screen keeps the mock until that is built.
Package the skill, the reviewer and the hooks as a personal plugin, install it in a clean sandbox, and move the conventions into a skill	Done	plugins/stream-frontend-kit v0.2.0: 6 skills, 3 reviewer agents, 2 hooks and 2 MCP servers; the conventions travel as skills generated from the rules. Validated in strict mode and installed into an empty configuration.
Run lint, types, unit tests, Playwright with axe and the performance budget in a CI pipeline	Out of scope	Out of scope by your decision: this demo app keeps no CI pipeline. Everything runs locally with npm run check, npm run e2e, npm run budget and npm run audit:devtools.
Write a one-page summary of your setup	Done	docs/learning-plan/SETUP_SUMMARY.md, and docs/PRODUCT_OVERVIEW.md for the product itself.

Throughout: reviewing AI output

Exercise proposed in the plan	Status	Evidence or what is left
Design-language conformance: only tokens and base components	Done	screen-lint, the screen reviewer and ESLint; nothing hard-coded in the screens.

Seeded defects: break a component on purpose and confirm a test or the reviewer fails	Done	Four guards removed on purpose, each made its tests fail; restored. Logged.
Accessibility: axe on every component and flow, plus a keyboard-only pass by hand	Partial	axe in both themes and at 320 px on every page; DevTools audit scores 100. The hand pass is yours.
States and traceability: every criterion maps to one test, every state to one story	Done	npm run traceability: 47 criteria and 17 states covered.
Security: no v-html with user text, no secrets in the bundle, nothing trusted unread	Done	vue/no-v-html is an error; plain-text tests; the secrets hook; independent reviewers.
A review log with at least one rejected or corrected artifact each week	Partial	The log has the corrections and the independent reviews so far. Add entries as weeks 2 and 3 happen.

Added after the plan (your requests)

Exercise proposed in the plan	Status	Evidence or what is left
Flows from the AdVise screenshots and the MVP criteria	Done	Launch per account (OEM rulebook, the account's own Salesforce budget, held items with reasons, one approval, partial failure, receipts), approvals in Campaigns, status filter.
Accounts with their own page and monthly budget; campaigns show their account	Done	/accounts and /accounts/:id (Campaigns, Launch, Changelog), Laravel accounts API with tests, account links on Campaigns.
Sandbox merged into Campaigns with full CRUD for simulated campaigns	Done	A tab per provider, Source column, create, edit and delete; Laravel PATCH and DELETE with tests.
Approvals as a table with approve and reject icons, inside Campaigns	Done	Approvals tab with the pending count in its label; the old address redirects.
Emergency stop moved out of the header into an Admin page (admins only) with the OEM rulebooks	Done	/admin with Emergency stop and OEM rulebooks tabs; the banner stays visible to everyone while writes are stopped.
UX and information design rule; tabs and accordions instead of stacked content	Done	.claude/rules/ux-information-design.md, docs/standards/ux.md, DsTabs and DsAccordion, tab in the URL.
Reviewers split by stack; skills, rules, agents and commands updated; Codex mirror generated	Done	frontend-code-reviewer and backend-code-reviewer; npm run sync:codex with a test that fails when the mirror is stale.
Accessibility and performance through DevTools, and updated packages (Node 26, Pest 5)	Done	npm run audit:devtools; TypeScript stays on 6.0 because typescript-eslint does not support 7 yet.
Documentation: product overview with roles, page expectations and diagrams; demo prompts with mockups; repository on GitHub	Done	docs/PRODUCT_OVERVIEW.md, prompts/, https://github.com/vanessaaristizabal-gl/demo-stream (semantic commits on main).